

Rural Vet Medicine Expiry & Emergency Swap Board

Take-home build challenge. This file is the source of truth for the project.

Read it fully before writing code. The original brief screenshots are in `docs/brief/`

— read those too; where this file and the screenshots disagree, **the screenshots win**.

1. The brief (verbatim)

Context

In rural districts, government veterinary dispensaries often operate in isolation across remote village clusters. Critical livestock medications, anti-venoms, and temperature-sensitive vaccines spoil unused on dispensary shelves because staff cannot easily see what neighboring centers have or need before batches expire.

The problem

Rural livestock assistants manage small dispensary clinics across separate villages.

Expensive

cattle anti-venom and foot-and-mouth vaccines expire on shelves in one clinic while neighboring villages face shortages during sudden outbreaks.

You will build a lightweight web board for field workers to flag expiring batches, post urgent drug requests, and confirm mutual transfers.

Must build

1. Log medicine inventory with vial count, batch number, cold-storage status, and expiry date.
2. Post urgent medicine requests tagged by urgency level and target village radius.
3. Match shortage requests automatically against nearby clinics holding matching surplus or expiring stock.
4. Generate a dual-party transfer pass with unique verification codes to log physical handoffs.
5. Provide a low-bandwidth offline draft mode that saves pending log entries locally until reconnected.

Constraints

- Must be fully deployed on a free hosting platform with zero paid API or database dependencies.

- Must function smoothly on a 360px mobile viewport without horizontal scrolling.
- Must pre-populate realistic mock data for at least five rural dispensaries upon first load.
- Must not require SMS gateways or complex OAuth logins; use lightweight clinic PIN identification.

How we judge it

1. Simplicity and speed of the matching and handoff confirmation workflow.
2. Sensible handling of edge cases such as double-claiming stock or network disconnects.
3. User interface clarity for low-digital-literacy field workers operating under stress.
4. Code hygiene, modular organization, and deterministic state management.
5. Pragmatic product decisions made to balance urgency with verification trust.

What to submit

- Public URL of the live, working web application hosted on a free tier platform.
- Public GitHub repository containing all source code, commit history, and setup instructions.
- A 1-page trade-off memo outlining trust safeguards, offline reconciliation logic, and

prioritized cuts.

Deadline

Sunday 6 September 2026, 3:47 pm IST. Time is the binding constraint on every decision below.

2. Decisions already made — do not relitigate

Stack: React + TypeScript + Vite, Supabase (free tier) for Postgres + Realtime, deployed to Vercel.

Why Supabase is permitted: the constraint bans *paid* APIs and *paid* databases. Free tier is not paid. Two must-builds require a shared remote store — "offline draft mode ... until **reconnected**" is meaningless without a remote to reconnect to, and "match against **nearby clinics**" requires reading other clinics' live inventory. A single-browser app cannot honestly satisfy either.

Auth: clinic code + 4-digit PIN, checked against a `clinics` table. No OAuth, no SMS, no email. PINs stored hashed. Session is a signed clinic id in `localStorage`. This is explicitly what the brief asks for; do not add Supabase Auth.

Styling: hand-written CSS, no UI framework, no web fonts. System font stack only — a webfont is a render-blocking network request for a user on 2G, and this app's users are the reason that matters. Keep the CSS in a small number of files with clear ownership.

Verify before committing: check Supabase's current free-tier inactivity/pause behaviour. If a project can sleep and greet a judge with a cold start, add a mitigation (a keep-warm ping, or a cached-state fallback on first load) and note it in the memo.

3. Domain model

Names below are the intended schema. Money quantities do not exist; **vials** are the unit.

clinics

id, code, name, village, district, lat, lng, phone, pin_hash, created_at

Seed at least 5. Use plausible Indian district/village names, real coordinates spaced 8–40 km apart so distance filtering produces interesting results.

drugs — controlled catalogue, not free text

id, name, form, unit, requires_cold_chain,
category

Rationale: free-text drug names destroy matching.

"FMD vaccine" / "Foot & Mouth vaccine" /

"F.M.D." never match each other. Field workers pick from a list. This is a trust safeguard as much as a UX one, and it should be called out in the memo.

batches

id, clinic_id, drug_id, batch_no, expiry_date,
cold_chain_ok, qty_reserved, status,
created_at

There is no qty_on_hand column. See below.

stock_movements — the ledger. This is the most important design decision in the project.

id, batch_id, delta, reason, actor_clinic_id,
client_id, client_ts, server_ts

reason ∈ received | dispensed | wasted |
expired | transferred_out | transferred_in |
correction

On-hand quantity is **derived**: SUM(delta) for a batch.
Never store and overwrite a running total.

Why this matters more than anything else here: two devices offline at once both recording "used 5 vials" must merge to -10, not to whichever number

synced last. Deltas are commutative; absolute totals are not. An app that stores `qty = 12` and overwrites it cannot be made correct offline, no matter how good its sync code is. Expose on-hand via a view or a computed selector, never a column.

requests

`id, clinic_id, drug_id, qty_needed, urgency, radius_km, needed_by, note, status, created_at`
`urgency ∈ routine | urgent | outbreak`
`status ∈ open | partially_filled | filled | cancelled | expired`

transfers

`id, request_id (nullable), batch_id, from_clinic_id, to_clinic_id, qty, sender_code, receiver_code, status, reserved_until, created_at, ...timestamps per transition`

`status ∈ proposed | accepted | in_transit | completed | declined | cancelled | expired | disputed`

events — append-only audit log

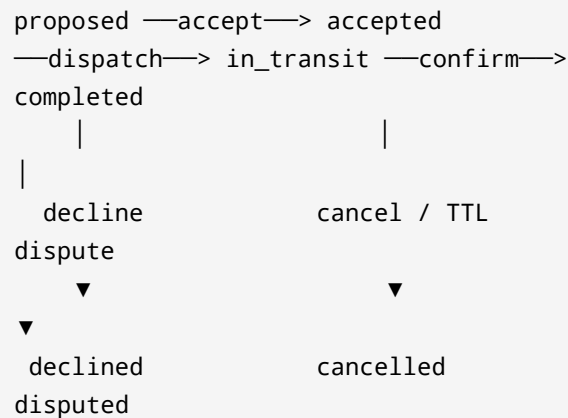
`id, entity_type, entity_id, type, actor_clinic_id, payload, client_ts, server_ts`

Nothing is ever hard-deleted anywhere in this system.
Corrections are new rows with a reason.
Shrinkage must remain visible — that is the whole
point of an audit trail in a system moving
expensive controlled stock between parties who do
not report to each other.

4. Transfer state machine

Implement transitions as **pure functions** with no I/O:

`(state, event) -> state`. Unit test them
directly. Judging criterion #4 names "deterministic
state management" explicitly — make it legible.



**Claim arbitration — judging criterion #2,
"double-claiming stock"**

Accepting a transfer must be a **single atomic server-side transaction** — a Postgres function

invoked via RPC, using `SELECT ... FOR UPDATE` on the batch row. It must:

1. Lock the batch row.
2. Recompute `available = SUM(deltas) - qty_reserved`.
3. Fail with a typed error if `available < qty`.
4. Otherwise increment `qty_reserved`, flip the transfer to `accepted`, issue both codes, set `reserved_until = now() + 24h`, write an `events` row.

Never do this check in the client. Two clinics tapping *Claim* on the last 4 vials within the same second is the exact scenario being graded — the loser must receive a clear, specific rejection ("Already committed to Marur dispensary 3 minutes ago"), not a silent failure or a stale success.

Reservation TTL

Accepted-but-never-dispatched transfers expire at `reserved_until` and release the reservation.

Without this, one clinic can lock scarce stock indefinitely by accepting and going quiet. Implement as a lazy sweep on read (cheap, no cron, no paid scheduler) — check-and-expire whenever a batch or board is queried.

Dual-party transfer pass — must-build #4

Both codes are issued **at accept time, not at dispatch**.

This is deliberate: the physical handoff happens on a road with no signal. Sender shows/reads their code, receiver reads theirs back, both devices record the exchange locally, and the two records reconcile when either reconnects.

- Matching codes on both sides → auto-complete.
- One side only → stays `in_transit`, visible as pending on both boards.
- Mismatch → `disputed`, surfaced to both clinics with the full event trail.

Codes are 6 digits, human-readable over a phone call. Generate server-side.

5. Offline behaviour — must-build #5, judging criterion #2

- Every mutating action is written to IndexedDB **first**, with a client-generated UUID (`client_id`), `client_ts`, and device id, then queued.
- The queue drains on reconnect. The server **dedupes on** `client_id` — replay must be safe. A

flaky connection retrying the same write three times must produce one movement, not three.

- **Clock skew is real.** Never order events across devices by `client_ts`. Server time orders; client time is for display and within-device sequencing only.
- **The UI must never claim a state the server hasn't confirmed.** A queued claim reads *"Waiting to send"*, never *"Confirmed"*. Honest pending states are a trust safeguard, not a UX detail — a worker who drives 30 km on a false confirmation loses trust in the system permanently.
- Contested actions (accepting stock) **cannot** be resolved on-device. Queue them, let the server arbitrate, and show the rejection clearly when it comes back. Own-clinic actions (logging dispensed vials, adding a batch) apply locally and immediately — they're never contested.
- Cache the last synced board so the app is useful with zero signal. Show a plain staleness marker: *"Last updated 2 hours ago."*

Build an explicit **offline toggle in dev/demo mode**. A judge must be able to cause a disconnect in one tap and watch the queue drain. Two of five judging criteria are about failure handling — make them demonstrable in seconds rather than described in prose.

6. Matching — must-build #3

Deterministic, explainable, no ML, no external API.

- Haversine distance from clinic coordinates.
- Filter: same `drug_id`, within the request's `radius_km`, available qty > 0, not expired.
- Rank: soonest expiry first, then nearest, then largest quantity.
- Prefer stock that is *expiring* — that's the entire premise of the product. Surface a match that solves both sides ("expires in 9 days, they need it in 3") above one that only solves one.
- **Always show why a match surfaced:** `11 km · expires in 9 days · 12 vials`. An opaque ranking is useless to a worker deciding whether to send someone on a motorbike.

7. UI rules

Audience: low-digital-literacy field workers, on cheap Android phones, in sunlight, under time pressure, during an outbreak. Design for the worst moment, not the demo.

- **360px viewport, no horizontal scroll, ever.** Test at 360px continuously, not at the end.
 - One primary action per screen. Large tap targets (min 44px).
 - Plain language. "Send 10 vials to Marur", not "Initiate transfer request".
 - The same action keeps the same name through the whole flow — the button that says *Claim* produces a status that says *Claimed*.
 - Status is never colour alone: colour **and** icon **and** text. Sunlight and colour-blindness both.
 - Expiry urgency is the visual spine of the inventory view — the one place to spend visual boldness. Everything else stays quiet.
 - Empty states instruct rather than apologise. Errors say what happened and what to do next.
 - Confirmation before anything irreversible or expensive; nothing else needs a modal.
-

8. Build order

Work top to bottom. The first four items are where the marks are — do not start UI polish until they are done and tested.

1. Repo, Vite scaffold, **deploy a placeholder to Vercel in the first hour** so the live URL exists

and never becomes a last-minute risk.

2. Supabase schema + RLS + seed data (5 clinics, drug catalogue, batches with staggered expiries, one already-contested batch for demo purposes).
3. Transfer state machine as pure reducers + unit tests.
4. Claim arbitration RPC + reservation TTL + dual codes.
5. Offline queue, idempotent replay, pending states.
6. Matching.
7. UI, mobile-first at 360px.
8. Seed realism pass, README with setup instructions, trade-off memo.
9. A short demo GIF in the README showing the double-claim rejection and the offline queue draining.

If time runs short, cut from #7 and #9 — never from #3–#5.

9. Out of scope — do not build

Maps and route planning. Photo uploads. Push notifications. SMS/IVR. Multi-language. Admin analytics dashboards. Real identity verification. Temperature sensor integration. Payment or

reimbursement between clinics. Role hierarchies beyond clinic-level access.

Each of these belongs in the memo's "prioritized cuts" section **with the reason it was cut**, not in the codebase. The memo asks for judgment, and naming what you deliberately didn't build is how you demonstrate it.

10. Working agreement

- **Commit frequently, with meaningful messages.**

Commit history is a stated deliverable and is the one thing that cannot be repaired at the end. Small, scoped commits.

- TypeScript strict mode on. No `any` in domain logic.

- Domain logic lives in pure modules with no React and no Supabase imports, so it can be tested in isolation. Judging criterion #4 is watching for exactly this separation.

- Unit tests for the state machine, claim arbitration, ledger arithmetic, and matching. Not for UI.

- No secrets in the repo. `.env.example` with documented keys; real values in Vercel env vars.

- README must let a stranger clone, configure, seed, and run in under five minutes.

- Propose a plan before major work and flag any deviation from this document rather than silently diverging from it.